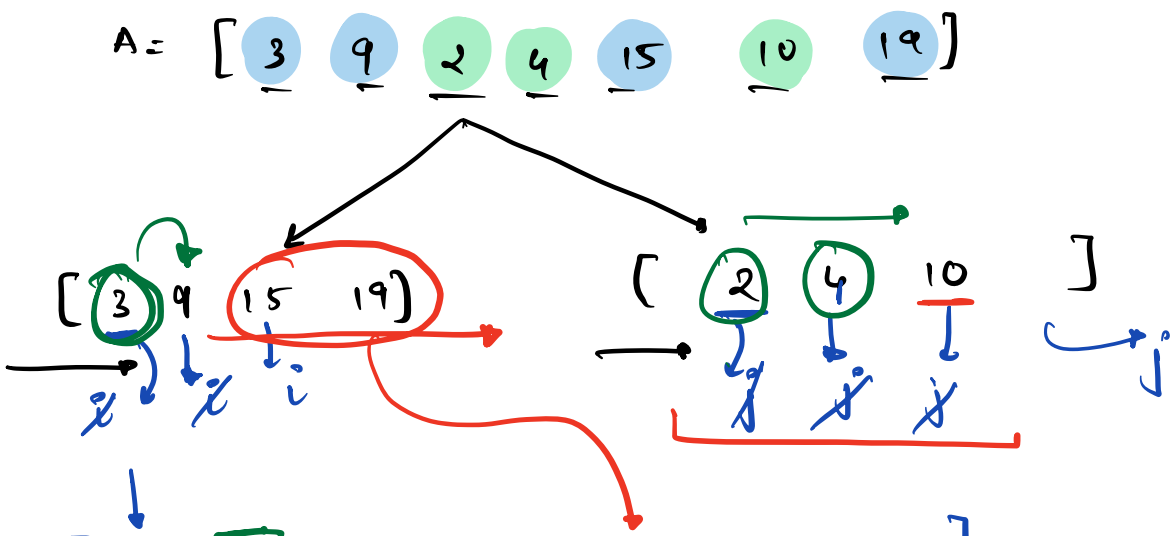
"Hello Everyone!"

Sorting - 01

(Q.1) Given an int array, where all odd elements are sorted and all even elements are sorted. Sort the entire array.

eg: A = [3 9 2 4 15 10 19]

→ [2 3 4 9 10 15 19]



→ [2, 3, 4, 9, 10, 15, 19]

→ Merge two sorted Array into one.

I/P: B[N] , C[M] O/P → A[N+M]

i=0 // B j=0 // C B[ ] C[ ]

to iterate over A[]

```
for k : 0 ⟶ (N+M-1)
{
    if (i == N)
    {
        A[k] = C[j]
        j += 1
    }
    else if (j == M)
    {
        A[k] = B[i]
        i += 1
    }
    else if ( B[i] <= C[j] )
    {
        A[k] = B[i]
        i += 1
    }
    else
    {
        A[k] = C[j]
        j += 1
    }
}
return A
```

[0] i [0] j

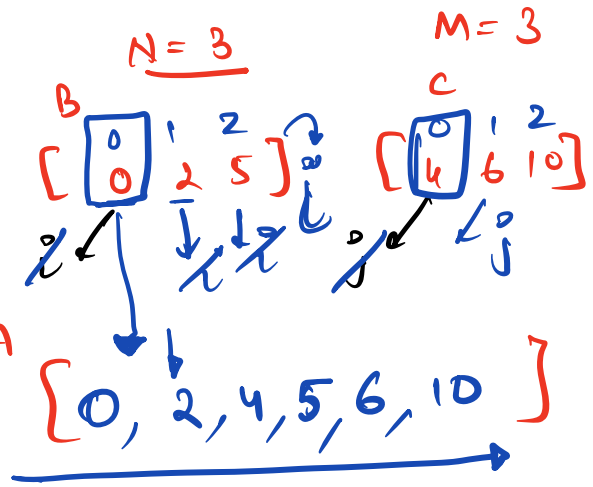
linear

TC = O(N+M)

SC = O(N+M)

linear

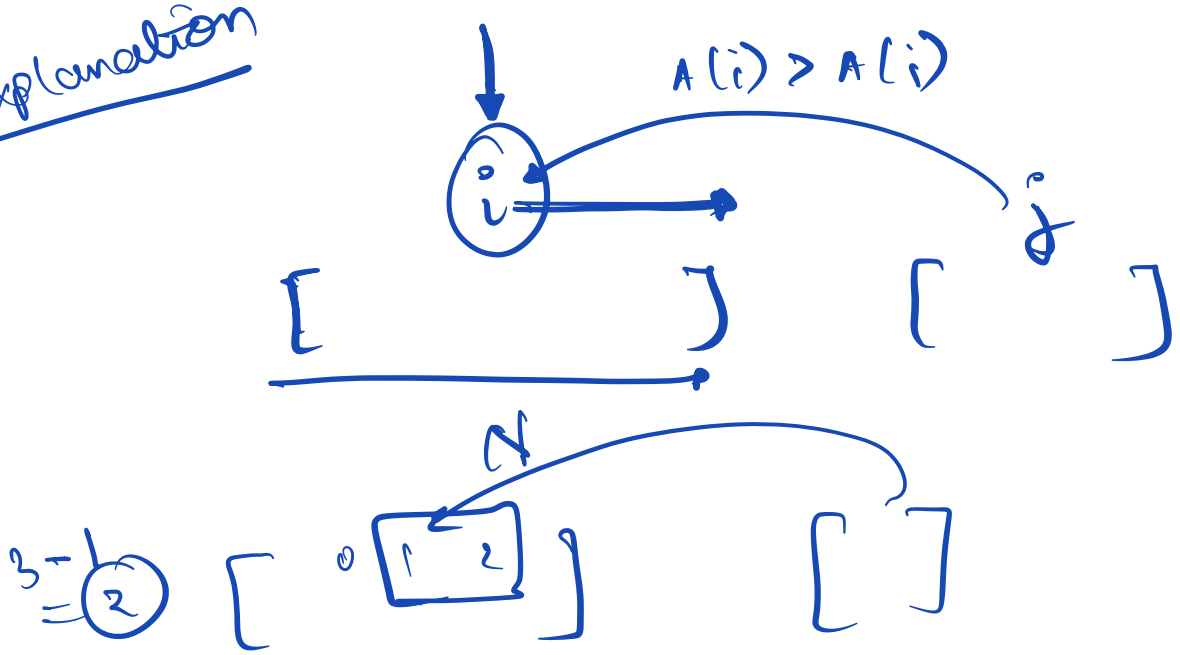
```
i=0 // B      j=0 // C
merge
{
  to iterate over A[]
  for k : 0 → (N+M-1)
  {
    if (i == N)
    {
      A[k] = C[j]
      j += 1
    }
    else if (j == M)
    {
      A[k] = B[i]
      i += 1
    }
    else if ( B[i] <= C[j] )
    {
      A[k] = B[i]
      i += 1
    }
    else
    {
      A[k] = C[j]
      j += 1
    }
  }
  return A
}
```

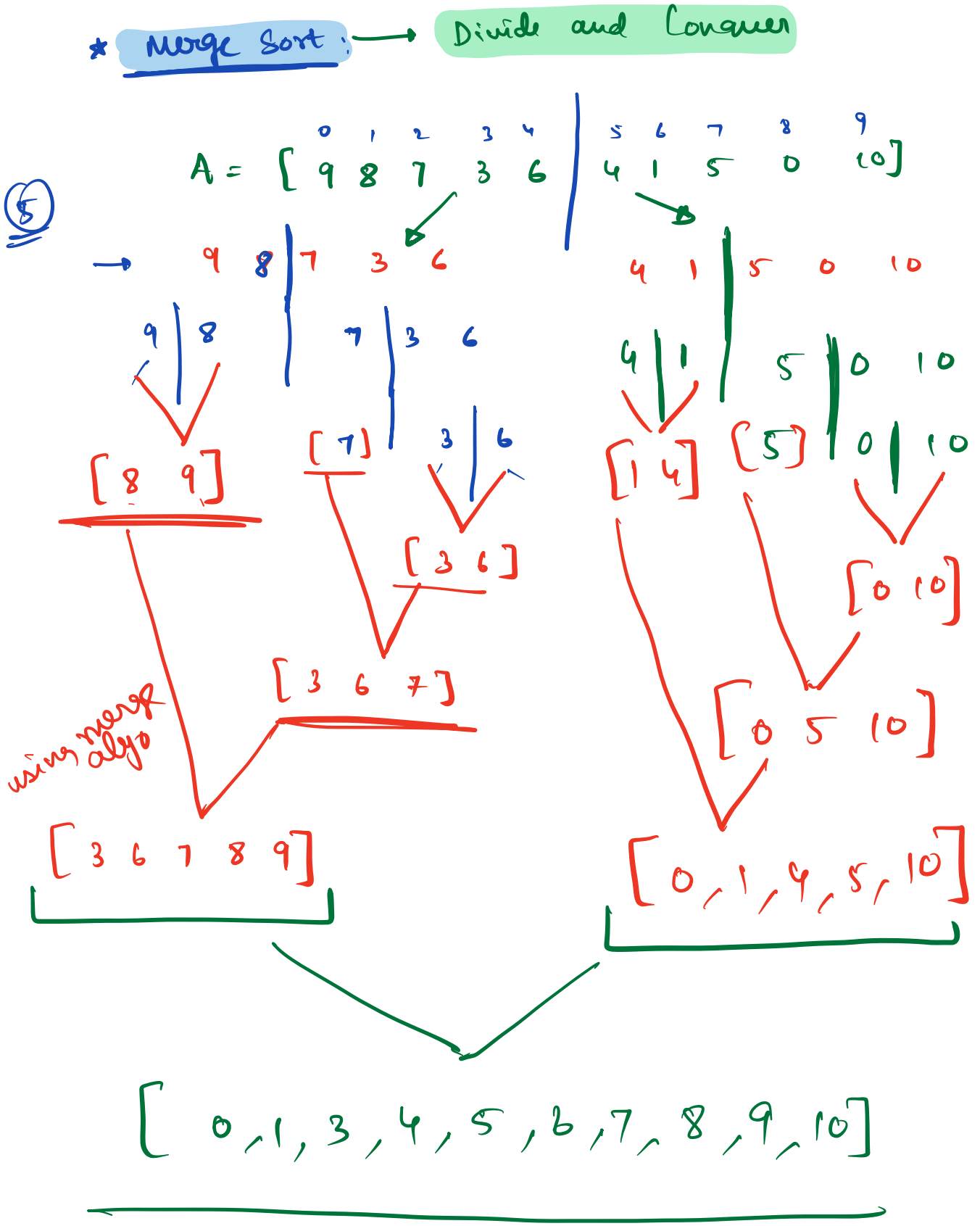


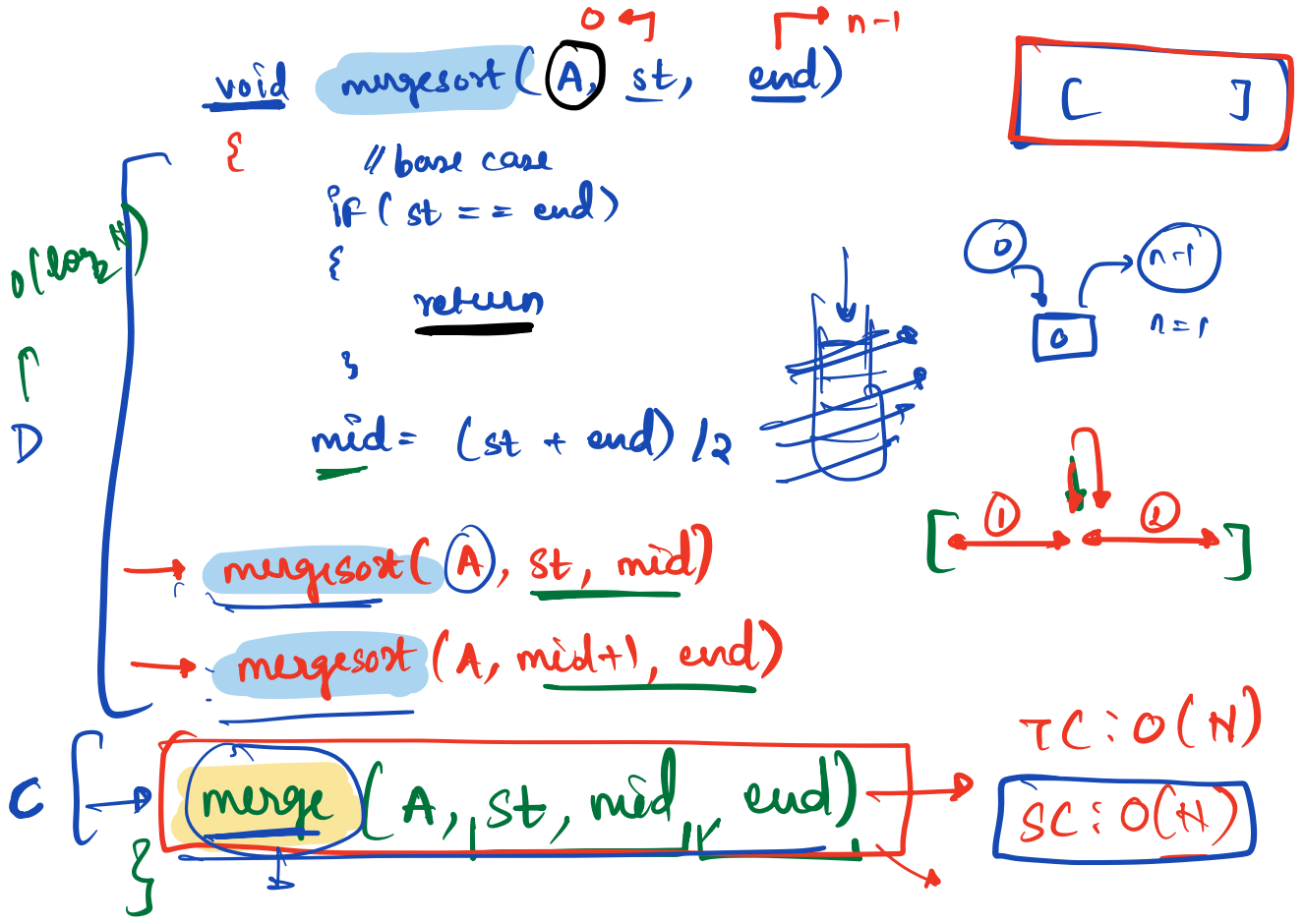
Right < left elem

count += (N - i)

explanation




* Merge Sort : → Divide and Conquer
A = [9 8 7 3 6 | 4 1 5 0 10]
0 1 2 3 4 5 6 7 8 9 10
9 8 | 7 3 6
4 1 | 5 0 10
9 | 8
7 | 3 6
4 | 1
5 | 0 10
[8 9]
[7]
3 | 6
[3 6]
[1 4]
[5]
0 | 10
[0 10]
[3 6 7]
[0 5 10]
using merge algo
[3 6 7 8 9]
[0, 1, 4, 5, 10]
[0, 1, 3, 4, 5, 6, 7, 8, 9, 10]

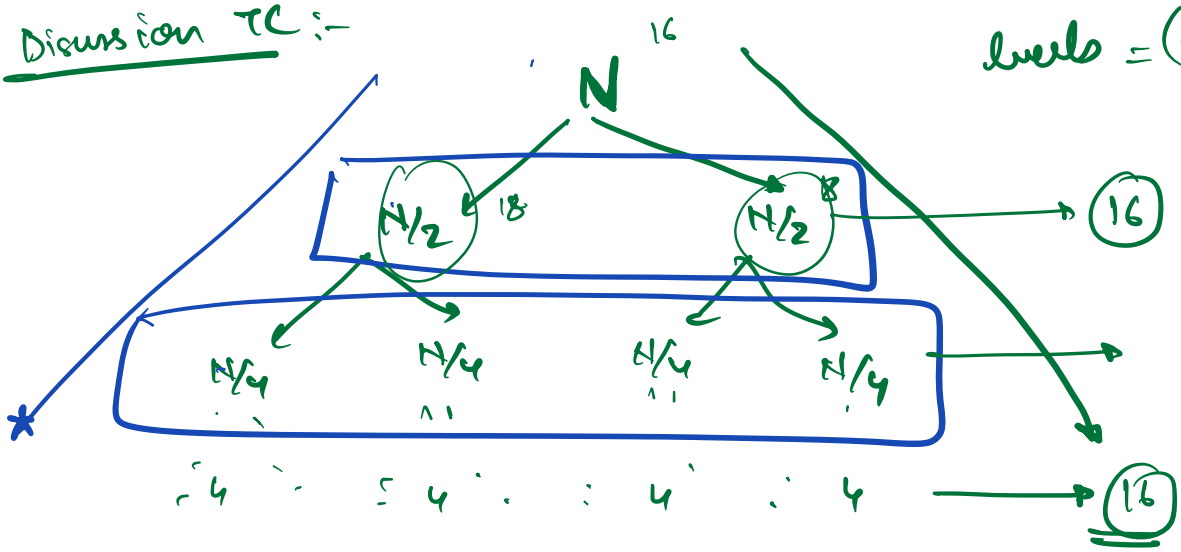


arr[] merge(A[], st, mid, end]

B // left → A[st] → A[mid]

C // Right → A[mid+1] → A[end]

Discussion TC :-

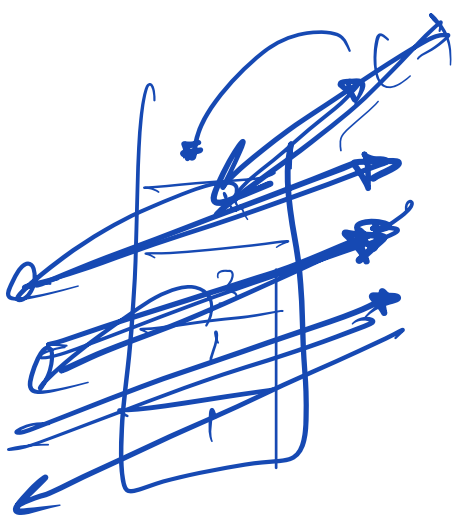


levels = $(\log_2 N)$

$N \xrightarrow{/2} N/2 \xrightarrow{/2} N/2 \longrightarrow 1$

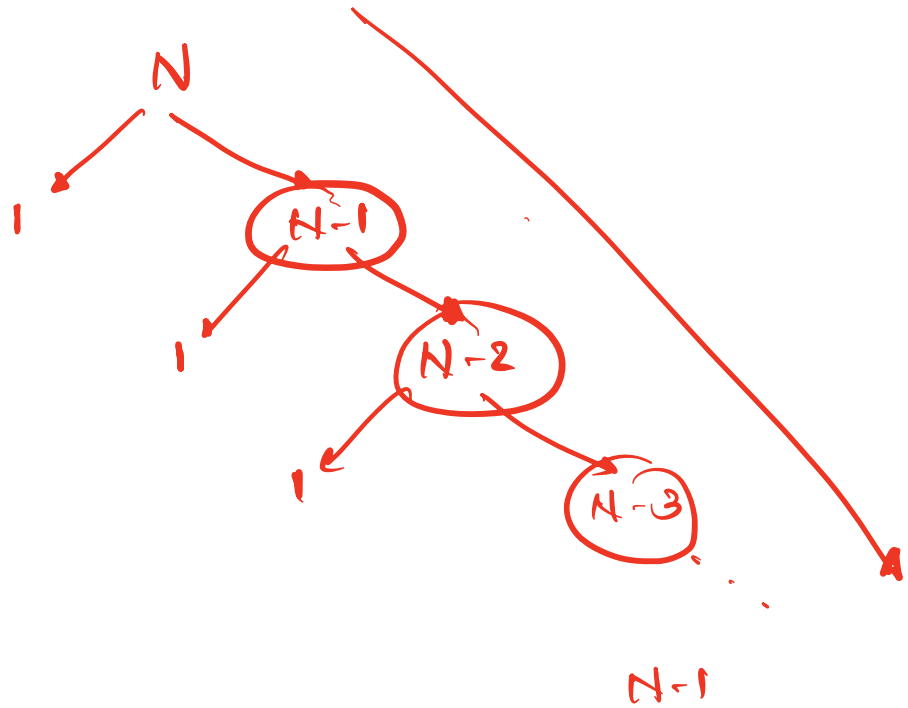
Total TC → $O(N * (\log_2 N))$

SC : $O(N + \log_2 N)$



why half?

([0][1 2 - - - - N-1 N])



N-1

O(N)

TC: O(N * O(N))

⇒ O(N^2)

TC :	O(N * $\log_2 N$)
SC :	O(N)

(Q.2) Given an int array, count the number of inversion pairs in the array.

Inversion pair → (i, j) ← indices

s.t

$i < j$ and $A[i] > A[j]$

eg1 :-

$A = [8, 3, 4]$ (indices 0, 1, 2)

→ i, j
(0, 1)
(0, 2)

eg2 :-

$A = [4, 5, 1, 2]$ (indices 0, 1, 2, 3)

(0, 2) (0, 3)
(1, 2) (1, 3)

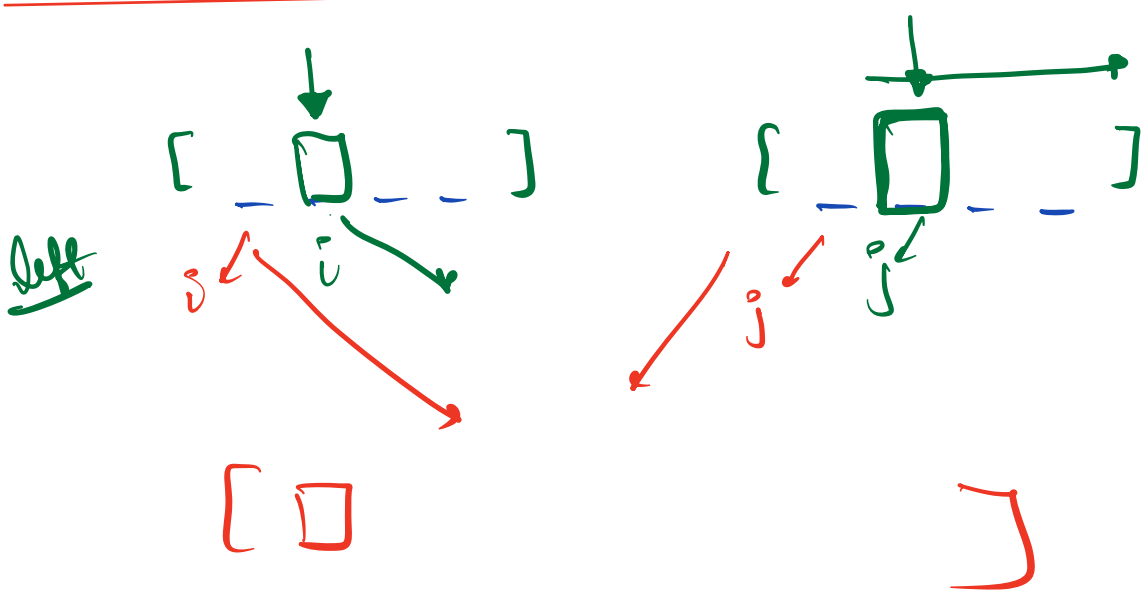
eg3 :-

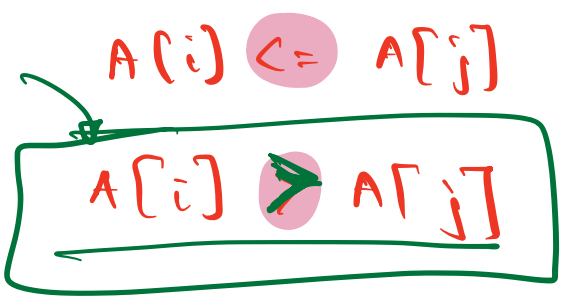
$A = [4, 4, 4, 4]$ (indices 0, 1, 2, 3)

→ 0

App 1 :- check all pairs $\forall i, j$,
check for inversion

$TC = O(N^2)$
$SC = O(1)$

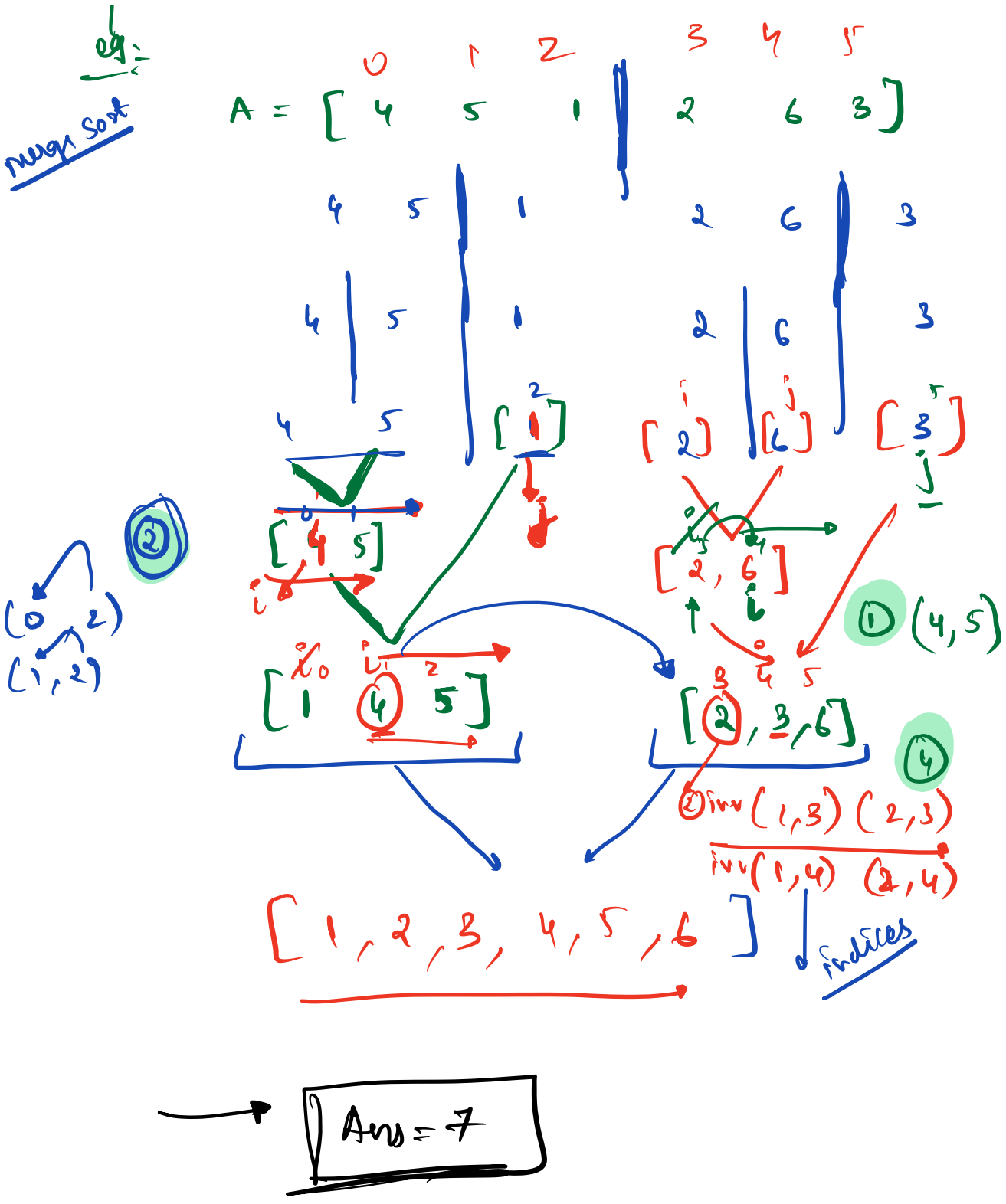




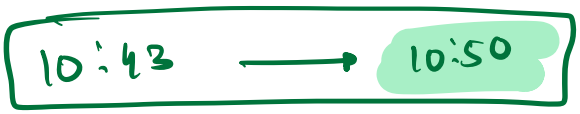
App 2 :- using mergeSort :- $TC = O(n * \log_2 n)$
$SC = O(n)$

eg:-

merge sort



Ans = 7


10:43
10:50

* Selection Sort

eg: A = [5 , 8 , 1 , 15 , 7 , 6 , 2] (indices 0 1 2 3 4 5 6)

(Q.1) Find max element in Array.

```
ans = A[0]
for i = 1 ⟶ (N-1)
{
    if (A[i] > ans)
    {
        ans = A[i]
    }
}
return ans
```

TC = O(n)

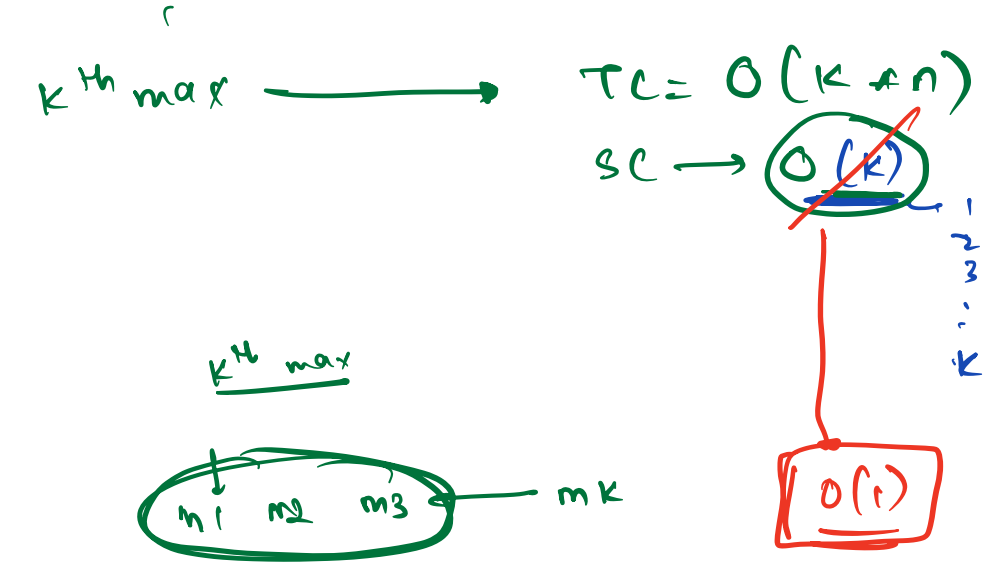
SC = O(1)

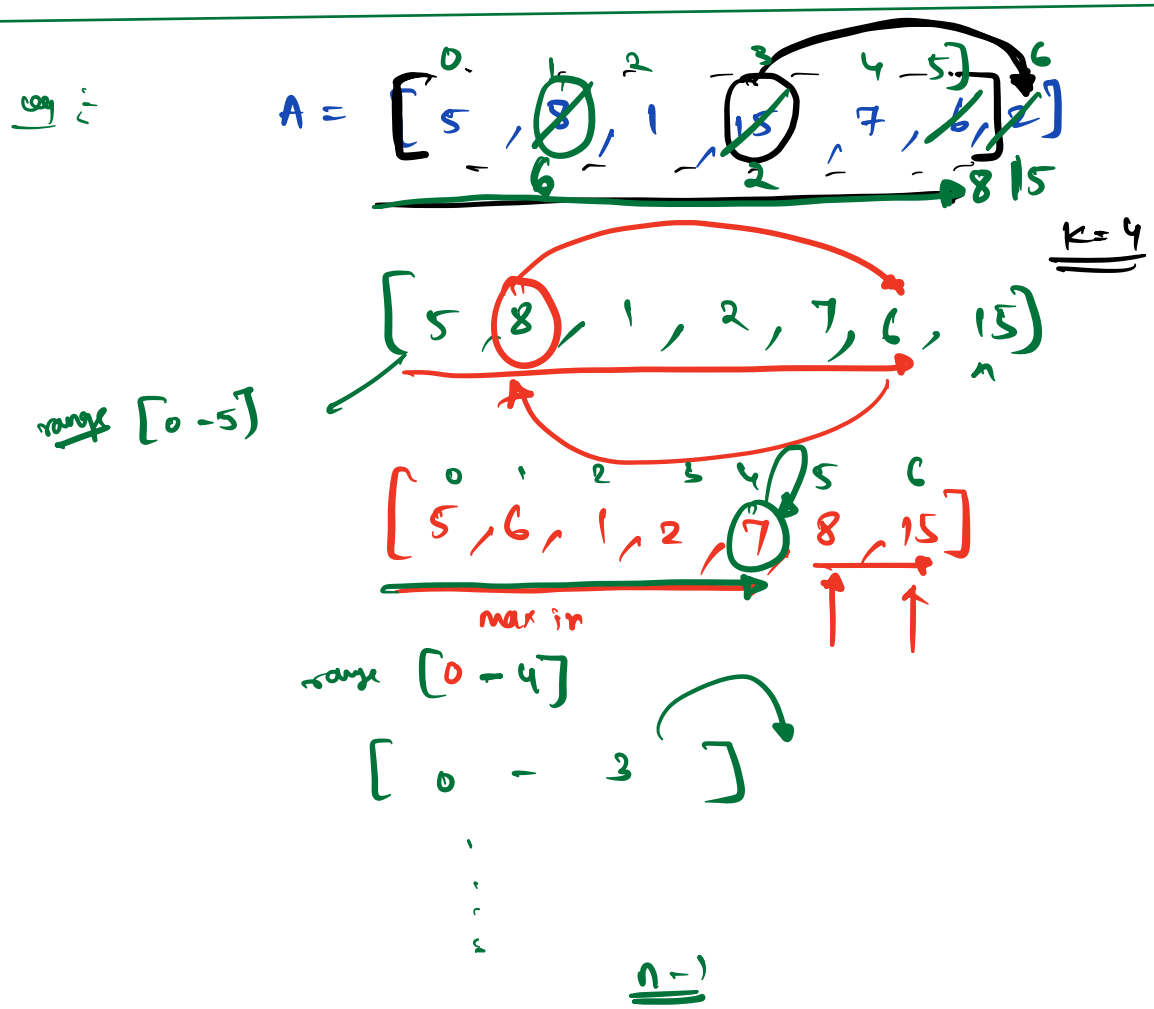
(Q.2) Find 2nd max elem ⟶ TC = O(2*n)
= O(n)

SC = O(1)

(Q.3) Find 3rd " ⟶ TC = O(3*n)
= O(n)

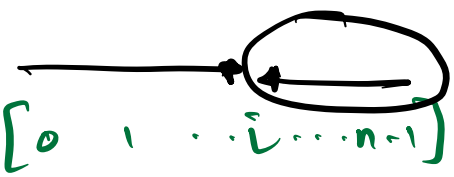
SC = O(1)


k^{th} max
TC = O(k * n)
SC → O(k)
1
2
3
.
.
k
k^{th} max
m1 m2 m3
mk
O(1)

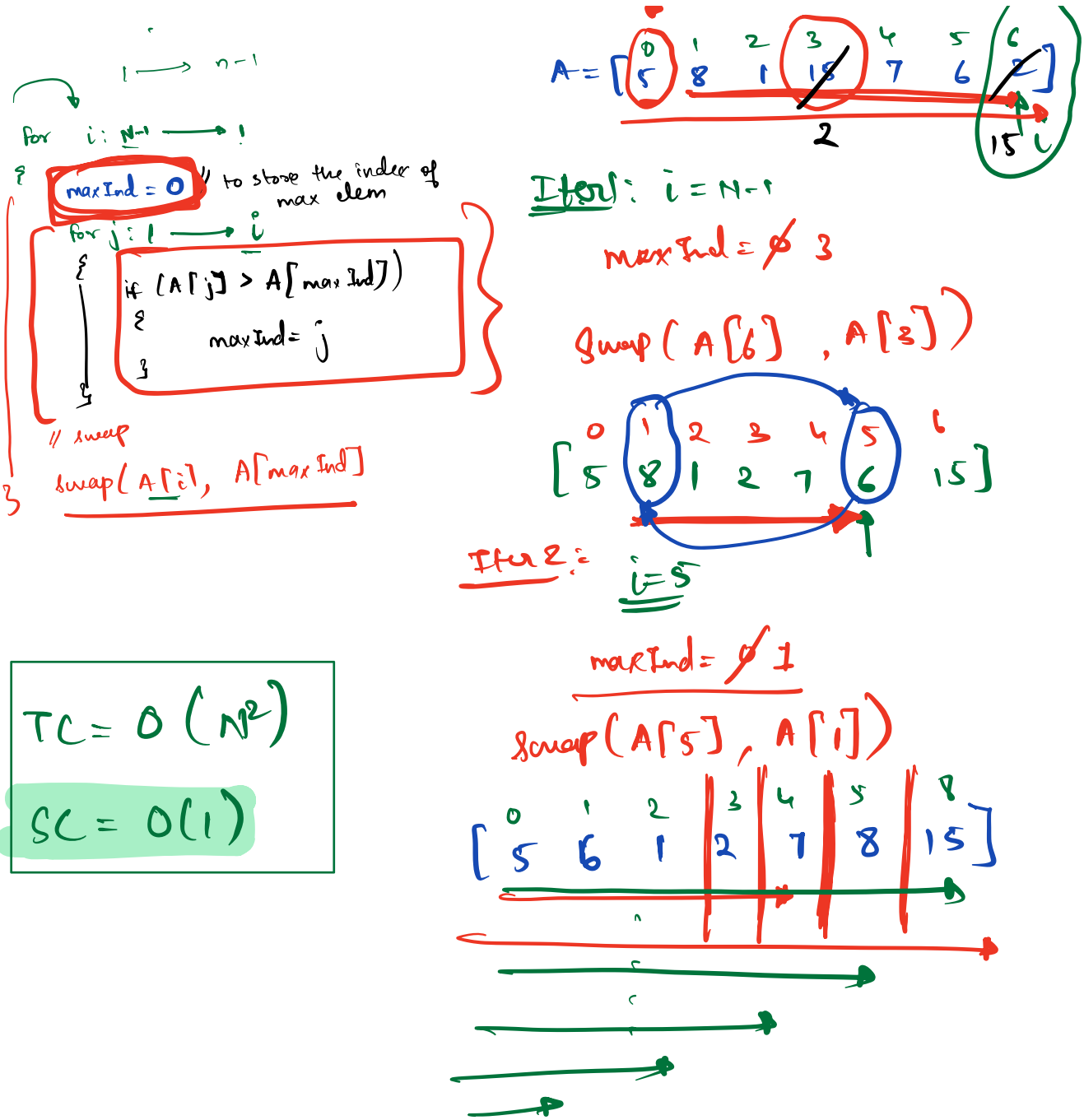

eg :-
A = [5, 8, 1, 15, 7, 6, 2]
0 1 2 3 4 5 6
6
2
8 15
k = 4
[5, 8, 1, 2, 7, 6, 15]
range [0 - 5]
[5, 6, 1, 2, 7, 8, 15]
0 1 2 3 4 5 6
max in
range [0 - 4]
[0 - 3]
n - 1

Logic : keep placing the max elem at its correct position in every iteration.

1st iter ⟶ 1st max
2nd iter ⟶ 2nd max
⋮
i ⟶ n-1




```
for i : N-1 ⟶ 1
{
    maxInd = 0 // to store the index of max elem
    for j : 1 ⟶ i
    {
        if (A[j] > A[maxInd])
        {
            maxInd = j
        }
    }
    // swap
    swap(A[i], A[maxInd])
}
```


1 ⟶ n-1
for i : N-1 ⟶ 1
{
maxInd = 0 // to store the index of max elem
for j : 1 ⟶ i
{
if (A[j] > A[maxInd])
{
maxInd = j
}
}
// swap
swap(A[i], A[maxInd])
}
A = [5 8 1 15 7 6 2]
0 1 2 3 4 5 6
2
15
Iter1 : i = N-1
maxInd = 3
swap(A[6], A[3])
[5 8 1 2 7 6 15]
0 1 2 3 4 5 6
Iter2 : i = 5
maxInd = 1
swap(A[5], A[1])
[5 6 1 2 7 8 15]
0 1 2 3 4 5 6

$TC = O(N^2)$

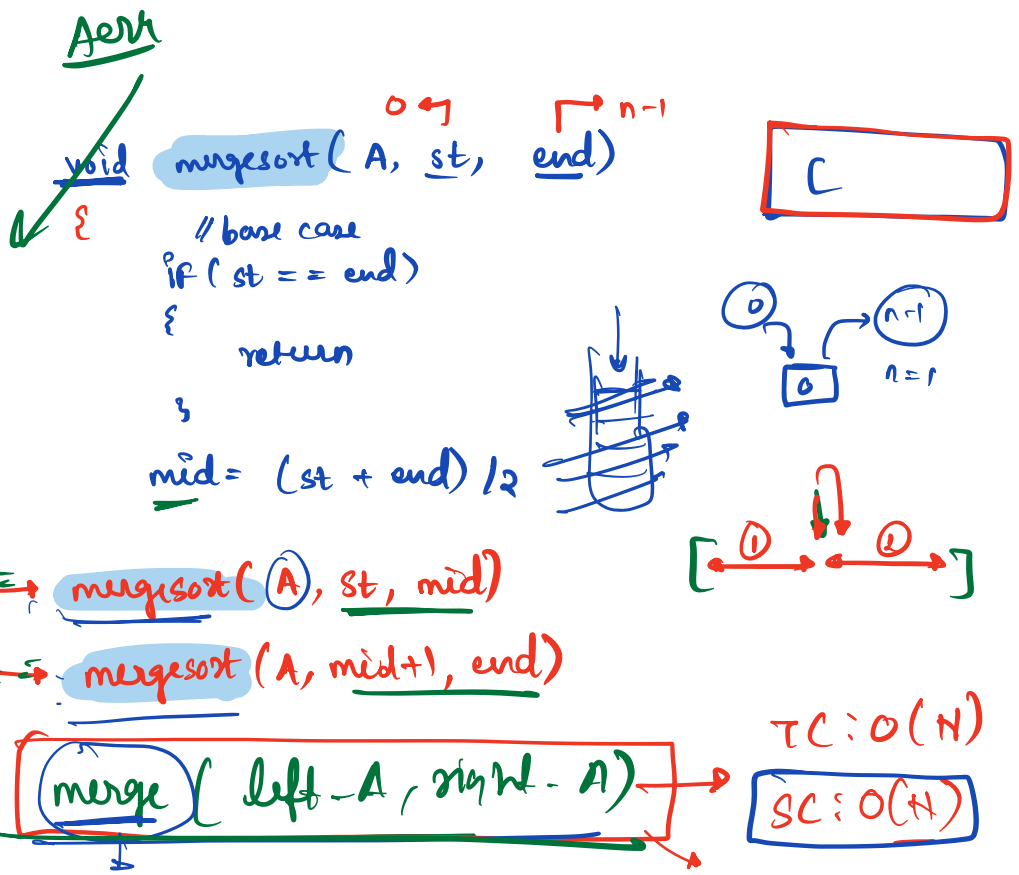
$SC = O(1)$

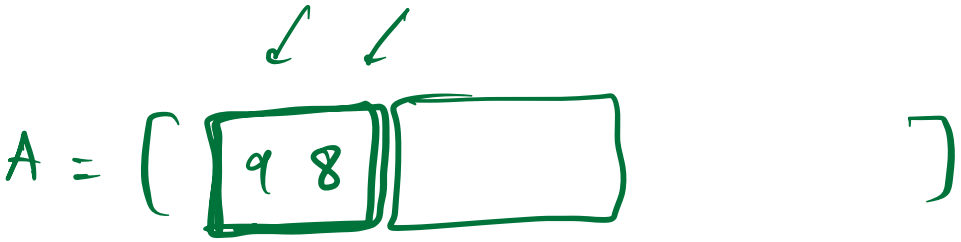
Anyone doubt

$$T(n) = 2 * T(n/2) + O(n)$$

a = 2
b = 2
d = 1

Case 1:




A = [9 8]